

COS711 Assignment 1: Test Notes

MLP on Protein Tertiary Structure: data preparation, core investigation, warmup and pruning

Kyle Liebenberg u22608789 Semester Test 2

How to use these notes

- Every section explains the *idea* first, then what *we* did, then what happened and *why*.
- Every section ends with a **cheat sheet** (the short version) and **likely questions** with answers. The questions are phrased generically, because the test has to work for all five datasets.
- Section 6 is a bank of general research-method questions, and Section 7 is a glossary for quick look-ups.
- Units: RMSD and RMSE are in **ångström (Å)**. Lower RMSE is better. “Accuracy” in the spec means our error metric, because our task is regression.

The answer template: use it for any “experiment” question

1. Hypothesis (what we predicted, and the theory behind it) → **2. Design** (what we changed, what we held fixed, how we measured) → **3. Result** (the key numbers) → **4. Verdict** (supported / partly / contradicted) → **5. Why** (connect the result back to theory; if we were wrong, say what our reasoning missed).

Contents

1 The whole project on one page	3
2 Data preparation	5
2.1 The data and the target	5
2.2 Cleaning: duplicates	5
2.3 Feature relevance	5
2.4 Outliers	6
2.5 Scaling	7
2.6 Splitting, stratification and k-fold cross-validation	7
2.7 Imbalance	8
2.8 Measuring performance	9
2.9 Likely questions: data preparation	10
3 MLP building blocks (refresher)	13
3.1 Our network	13
3.2 One training step	13
3.3 Learning rate and schedules	13
3.4 Weight initialisation	13
3.5 Overfitting and early stopping	14
3.6 Regularisation in general (the axis we did not test)	14

3.7	Baselines and noise	14
3.8	Likely questions: building blocks	15
4	Core investigation (the axes)	16
4.1	What we did and how	16
4.2	Axis A: Optimisers	16
4.3	Axis B: Architecture (depth vs width)	17
4.4	Axis C: Activation functions	18
4.5	The axes we did not run (in case they are asked)	19
4.6	Final test evaluation	20
4.7	Choosing the Part 2 network	20
4.8	Likely questions: core investigation	21
5	Research investigation: warmup and pruning	24
5.1	The question, in plain words	24
5.2	The three papers	24
5.3	Key concepts	24
5.4	Fixed setup	25
5.5	Step 1: does warmup raise the tolerable learning rate? (H1)	25
5.6	Step 2: choosing the conditions (rules fixed in advance)	25
5.7	Step 3: prune, fine-tune, compare (H2 and H3)	26
5.8	Alternative explanations we ruled out (confounders)	26
5.9	Extensions E1, E2, E3	27
5.10	Conclusion and limitations	28
5.11	The spec's other extensions (not done, but could be asked)	28
5.12	Likely questions: warmup and pruning	29
6	General research-method questions	31
7	Glossary	32

1 The whole project on one page

The task

In the CASP protein-folding competition, scientists measure a protein's true 3D shape and keep it secret. Prediction programs then produce many **candidate shapes** (“decoys”) for that protein. Each row of our dataset is one decoy, described by **9 physicochemical measurements (F1–F9)**, such as surface areas and how much water-repelling surface is exposed. The target is the decoy's **RMSD**: how far its atoms are from the true shape, in ångström. 0 means perfect, around 2 means very close, and around 15 means essentially wrong.

Our MLP takes the measurements in and predicts the RMSD out. That makes it a **regression** task. It is useful because for a new protein the true shape is unknown, so a model like this can rank thousands of candidates and point to the likely best ones.

What the spec asked, and what we did

Part	Spec asks	We did
Data preparation	Feature relevance, outliers, scaling, splitting, imbalance	Removed duplicates, dropped F2, log + clip, z-score, stratified 80/20 split with 5-fold CV, per-bin evaluation
Core investigation	A baseline, then ≥ 2 axes with k-fold CV, a hypothesis before each	Baseline 2×64 MLP, then 3 axes: optimisers, architecture, activation functions
Research investigation	Warmup vs no warmup at comparable accuracy, magnitude pruning, identical fine-tuning, ≥ 3 seeds, interpret the mechanism	Conditions A, B, C; global magnitude pruning at 0–98%; 5 seeds; extensions E1, E2, E3

The numbers worth remembering

Rows	45,730 raw $\rightarrow$ 44,019 after removing 1,711 exact duplicates
Split	35,215 train / 8,804 test (80/20), stratified on 3 Å RMSD bins, seed 42
Inputs	8 features (F2 dropped because $F2 = F1 \times F3$ exactly)
Baseline	2×64 ReLU MLP, Adam 10^{-3} : CV RMSE 4.09 Å , $R^2 = 0.555$
Reference models	Predict the mean 6.14 Å; linear regression 5.19 Å; gradient-boosted trees 4.13 Å
Best model	4×256 ReLU, Adam 10^{-3} : CV 3.81 Å ; test 3.78 ± 0.03 Å , $R^2 = 0.621$
Part 2 network	4×256 ReLU, SGD + momentum 0.9, 100 epochs, cosine decay, warmup = 5 epochs
LR tolerance	No warmup: max stable LR 0.005 . With warmup: trains up to 0.2 ($\geq 20\times$)
Conditions	A = no warmup, LR 0.005. B = warmup, LR 0.2. C = warmup, LR 0.005 (control)
Headline	At $\geq 70\%$ sparsity, B loses 0.22–0.27 Å less than A after fine-tuning, in 5/5 seeds. $C \approx A$

The research answer in three sentences

Warmup let the network train at a learning rate at least 20 times larger without blowing up. At matched unpruned accuracy, the network trained at that large learning rate was clearly more robust to magnitude pruning. Warmup at the *small* learning rate gave no benefit, so warmup helps pruning

robustness **only indirectly, by unlocking a larger learning rate**, exactly as the hypothesised mechanism says.

How we worked (this is what “a fair experiment” means)

- **Hypotheses first.** Every hypothesis and decision rule was written down and committed to git *before* the experiment ran, so we could not adjust the prediction to fit the result.
- **One variable at a time.** Everything except the setting under study stays at the baseline.
- **Same data for everyone.** Every configuration sees exactly the same 5 folds.
- **Repeat and measure the noise.** 3 seeds $\times$ 5 folds in Part 1, 5 seeds in Part 2. A difference only counts if it is bigger than the noise *and* points the same way in (nearly) every seed.
- **The test set is locked.** It is used once, at the end. Every choice is made on training data.

2 Data preparation

The spec makes data preparation compulsory and names five topics: **feature relevance**, **outliers**, **scaling**, **splitting** and **imbalance**. One rule sits behind every decision:

The one rule: no data leakage

Once the test set is split off, every statistic (correlations, percentiles, means, standard deviations) is computed from the **training data only**. If the test data influenced any of these, information about the test set would leak into the model, and the test score would look better than the model really is. The same applies inside cross-validation: the preprocessing is re-fitted on each fold's training rows only.

2.1 The data and the target

- 45,730 rows, 9 features, no missing values, all values non-negative. F8 is a whole-number count with 341 different values, so it is treated as continuous.
- Six features are strongly right-skewed (skew > 1): a long tail of large values. **F7 is extreme** (skew 21): its maximum is 105,948 while its median is 3,841.
- **The target RMSD is bimodal**, meaning it has two peaks: a tall peak of good decoys at 1–3 Å and a broad hump of poor decoys at about 11–17 Å, with a dip in between (Figure 1a). It ranges from 0 to 21 Å.

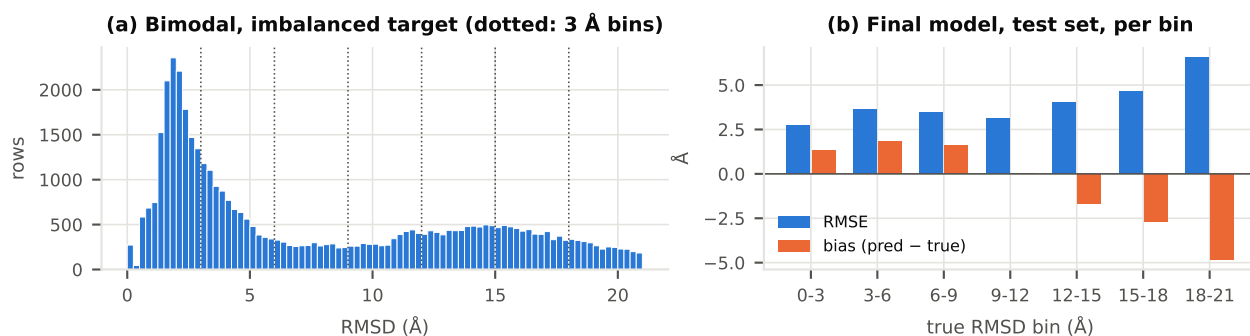


Figure 1: (a) The target is bimodal and imbalanced; the dotted lines are the 3 Å bins. (b) The best Part 1 model (4×256, ReLU, Adam 10^{-3} , trained in notebook §3.5) on the test set, per bin. Blue = RMSE in that bin. Orange = bias (prediction – truth): positive means it guesses too high, negative means too low.

2.2 Cleaning: duplicates

1,711 rows (3.7%) were exact copies (same features *and* same target). We removed them **before** splitting. If a row and its copy landed on opposite sides of the split, the model would be tested on a row it had already memorised. That is leakage, and it makes the score optimistic.

A further 83 rows share identical features but have *different* RMSD values. These were kept: they are genuine noise in the labels, and no model can predict both values.

2.3 Feature relevance

The question: is any feature useless? We used three measures, each catching a different kind of relationship:

Measure	What it detects	Weakness
Pearson r	A straight-line relationship (-1 to $+1$)	Misses curves. $r \approx 0$ does not mean “no relationship”
Spearman ρ	A consistently rising or falling relationship (Pearson on the ranks)	Misses U-shapes
Mutual information (MI)	Any relationship: how much knowing the feature reduces uncertainty about the target	Has no fixed scale, so we added a random-noise column as the “useless” reference (MI = 0.000)

We also did a model-based check: gradient-boosted trees with 5-fold CV, leaving out one feature at a time to see how much accuracy dropped.

Findings.

1. **No feature is uninformative.** Every feature’s MI (0.18–0.35) is far above the noise column’s 0.000, even though five features have almost zero Pearson correlation. **F7 has $r = -0.003$ but the highest MI (0.350).** A correlation-only filter would have thrown away our most informative feature. The relationships are non-linear, which is also the argument for using an MLP instead of linear regression.
2. **F2 is redundant: $F2 = F1 \times F3$ exactly.** F3 is the *fraction* of exposed area that is non-polar, so non-polar area ($F2$) = total area ($F1$) $\times$ that fraction. It adds no new information, so we dropped it.
3. **F5 looked redundant but is not.** $F5 \approx 138.5 \times F1$ (correlation 1.00), and the tree model barely noticed removing it. But the small deviation, the ratio $F5/F1$ (roughly the average mass of the exposed residues), carries information (MI 0.089). When we re-checked with *the MLP itself* over 5 seeds, the model with F5 beat the model without it in **5/5 seeds** (4.10 vs 4.15 Å). So F5 was restored.
4. Final decision: **8 inputs** (all except F2).

Uninformative vs redundant (a likely test distinction)

Uninformative: no relationship with the target at all (MI about the same as random noise). Remove it because it only adds noise. **Redundant:** it is related to the target, but the same information is already in other features. Removing it loses nothing. We found *no* uninformative features and *one* redundant one (F2).

Lesson from F5: “redundant” depends on the model. After the log transform, $\log F5 - \log F1$ is a simple difference that an MLP’s first layer can compute with two weights, while a tree splits on one feature at a time and struggles with differences. Always check feature decisions with the model you will actually use.

2.4 Outliers

Textbook rules. The **IQR rule** flags a value as an outlier if it lies more than $1.5 \times \text{IQR}$ below the 25th percentile or above the 75th percentile (IQR = the spread of the middle 50%). The **z-score rule** flags values more than, say, 4 standard deviations from the mean. Both assume a roughly *symmetric* distribution.

The problem. On right-skewed measurements such as areas, these rules mostly flag the *natural long tail*, not errors. For example, the IQR rule flagged 1,864 F8 values. After a **log1p** transform, which squashes long tails, those flags almost disappeared for F1, F4, F6 and F8. They were normal spread, not mistakes.

F7 is the real exception. Even after the log it has extreme values at both ends. But the rows with extreme F7 have ordinary RMSD values and ordinary other features. The anomaly is in F7 alone.

Our strategy: transform and clip, don't delete.

1. $\log_{1p}(x) = \log(1 + x)$ on the skewed features. The “+1” is needed because F7 and F8 contain zeros.
2. **Clip (winsorise)** every feature to its training-set **0.5th–99.5th percentile**. A value above the upper limit is replaced by the limit. The row is kept; only the extreme input value is capped, so it says “very high” instead of “50 standard deviations high” and cannot dominate a neuron's weighted sum.

Why not delete the rows? (a) Their targets are valid and informative. (b) In real use we cannot refuse to predict for a decoy with an odd F7, so the model must cope with such inputs anyway. (c) Deleting rows from the test set would remove hard cases and flatter the score.

The target is never treated as an outlier. A large RMSD is a genuinely bad decoy, which is exactly what we want to predict.

2.5 Scaling

Pipeline, fitted on training data: **select 8 features** $\rightarrow$ **log1p the skewed ones** $\rightarrow$ **clip** $\rightarrow$ **standardise**.

Standardisation (z-score): $z = (x - \mu)/\sigma$, using the training mean μ and standard deviation σ . Afterwards every feature has mean 0 and standard deviation 1. Test data is transformed with the *same* training μ and σ .

Why standardise at all?

- **One learning rate for all weights.** Raw F5 is around 10^6 while F3 is around 0.3. Unscaled, the weights for different inputs would need learning rates that differ by orders of magnitude.
- **Initialisation assumptions.** Xavier and He initialisation assume inputs with roughly unit variance, so signals neither explode nor vanish as they pass through the layers.
- **Saturating activations.** Sigmoid and tanh go flat (gradient ≈ 0) for large inputs. Standardised inputs start them in their responsive middle region.

Why z-score rather than min-max (scaling to 0–1)?

- **Zero-centred.** Min-max values are all positive. If every input to a neuron is positive, all of its weight gradients share the same sign, so the weights can only all go up or all go down together, which causes zig-zagging and slow training.
- **Robust to extremes.** Min-max depends on the single largest and smallest value. F7's maximum would squeeze 99% of its values into a tiny slice of $[0, 1]$.

The target is standardised for training too, so the loss is on a unit scale. Predictions are converted back to ångström before *any* metric is computed.

2.6 Splitting, stratification and k-fold cross-validation

Stratification

Stratification means every part of a split gets the same mix as the whole. With classes (e.g. 10% spam) it guarantees each part is about 10% spam. Our target is a number, not a class, so we **bin it temporarily** into seven 3 Å buckets (0–3, 3–6, ..., 18–21) and stratify on those bucket labels. The model still predicts the actual number. Train and test bin shares then match to within 0.01 percentage points. Without stratification, a test set or fold could by bad luck get too few of the rare high-RMSD decoys, and its score would be misleading.

k-fold cross-validation

The problem it solves: to compare settings (Adam vs SGD, 2 vs 4 layers) we need a score on unseen data. We cannot use the test set, because choosing by test score means the test set stops being a fair final exam. One single validation chunk is fragile: the result depends on which rows happen to land in it.

The idea: cut the training data into k equal folds. Run k rounds. In each round one fold is held out for scoring and the model trains on the other $k - 1$. Every row is scored exactly once. Report the **mean $\pm$ standard deviation** of the k scores.

Round	Train on rows	Score on rows
1	3–10	1, 2
2	1–2, 5–10	3, 4
3	1–4, 7–10	5, 6
4	1–6, 9–10	7, 8
5	1–8	9, 10

Example: 10 rows, $k = 5$. Scores 4.05, 4.12, 4.08, 4.15, 4.10 give **4.10 $\pm$ 0.04**. If another setting scores 4.11 ± 0.04 , the difference is far smaller than the noise, so neither is better.

Why it helps: less luck (5 scores instead of 1); the standard deviation shows the noise level; no data wasted; and the test set stays untouched. We used $k = 5$, and **every configuration saw exactly the same folds**, so differences come from the configuration and not from easier folds. $k = 5$ is a standard trade-off: larger k gives more training data per round but costs k trainings per configuration.

Common misconception: $k = 5$ is *not* linked to the 80/20 split, and no fold is the test set. The test set is removed first and never takes part in cross-validation.

The early-stopping split inside each fold

Early stopping watches the error on separate data after each epoch and keeps the best epoch (Section 3.5). That “separate data” must not be the fold we report, or we would be choosing the stopping epoch with the very data we score, which is optimistic. So each round splits three ways:

train 80% (35,215 rows)		test 20%
fold’s training part: 4 folds		validation
fit 90%: weights learn here	stop	

- **Fit** ($\approx 25,300$ rows): the weights learn here, and the preprocessing is fitted here.
- **Stop** ($\approx 2,800$ rows): watched every epoch, only to decide when to stop.
- **Validation** ($\approx 7,000$ rows): scored and reported, never used during training.
- **Test** (8,804 rows): locked until the very end.

Limitation: the data has no protein IDs. Many decoys come from the same protein and look alike, so near-identical decoys can sit on both sides of a split. Absolute scores may therefore be slightly optimistic, but *comparisons* between settings are still fair, because every setting uses the same splits.

2.7 Imbalance

For regression, “imbalance” means some ranges of the target are much rarer than others. Our 0–3 Å bin holds 34% of rows, the 18–21 Å bin only 7% ($4.9\times$ fewer). A model trained with plain squared error is pulled towards the common regions.

Stage	How we handle it
Training	Stratified CV folds and stratified early-stopping split: every fold sees every RMSD range in the right proportion
Testing	The test split is stratified on the same bins, so the final score is a fair picture
Evaluation	Overall RMSE, MAE and R^2 plus RMSE and bias per bin , so failures in rare ranges cannot hide behind a good average

Why per-bin reporting matters. A model that always predicts the mean has an overall RMSE of 6.14 Å, but per bin its error ranges from 0.9 Å (6–9 Å, close to the mean) to 11.6 Å (18–21 Å). One number hides *where* a model fails. Our best model (Figure 1b) has 2.75 Å error in the common 0–3 Å bin but 6.56 Å in the rare 18–21 Å bin.

Bias and regression to the mean. Bias = average of (prediction – truth) in a bin. Our model over-predicts the good decoys (+1.3 Å in 0–3 Å) and under-predicts the bad ones (–4.8 Å in 18–21 Å). When unsure, it plays safe and pulls its guesses towards the middle of the data.

Inverse-bin-frequency loss weights: tried and rejected

What it is: each row’s squared error is multiplied by $1 / (\text{how common its bin is})$, so rows in rare bins count more (a row in the 7% bin weighs about $4.9\times$ a row in the 34% bin) and every bin contributes equally to the loss. It is the regression version of *class weights* in classification.

Why we tried it: it is the standard fix for imbalance, and the rare bins had the worst errors.

Why we rejected it:

1. It moved errors around instead of removing them. Rare bins improved by 0.15–0.54 Å, but the common 0–3 Å bin got 1.02 Å worse. **Overall RMSE rose from 4.09 to 4.31 Å.**
2. It did not fix the real problem. The under-prediction of bad decoys barely changed (bias $-5.6 \rightarrow -5.4$ Å); mostly it shifted *all* predictions upwards. The model under-predicts bad decoys because the features cannot tell a very bad decoy from a moderately bad one, not because the loss ignores them. Re-weighting cannot add information that the features do not contain.
3. The good, near-native decoys matter most in practice, and weighting made exactly those worse.

2.8 Measuring performance

All metrics are computed in ångström, on data the model did not learn from.

Metric	How it is calculated	What it means and why we use it
RMSE	square root of the mean of $(\text{pred} - \text{true})^2$	Typical error size, with big mistakes counting extra. Our headline metric: same units as the target, and it is the square root of MSE, which is exactly what the network is trained to minimise
MAE	mean of $ \text{pred} - \text{true} $	The plain average error. Always $\leq$ RMSE; a big gap means a few large errors
R^2	$1 - (\text{sum of our squared errors} \div \text{sum of squared errors when always predicting the mean})$	Fraction of the target’s variation explained. 1 = perfect, 0 = no better than guessing the mean, < 0 = worse. Gives a unit-free sense of “good”

Worked example. True values 2, 10, 18; predictions 4, 9, 12. Errors +2, –1, –6; squared 4, 1, 36. $\text{RMSE} = \sqrt{41/3} \approx 3.70$. $\text{MAE} = 9/3 = 3.0$. The mean of the truth is 10, and predicting 10 everywhere gives squared errors $64 + 0 + 64 = 128$, so $R^2 = 1 - 41/128 \approx 0.68$.

CV score vs test score. The *CV score* is the average over the validation folds; it is measured on training data and used to *choose* between configurations, many times. The *test score* is measured once, on the locked test set, after all choices are made. Our best model: CV 3.81 Å, test 3.78 Å. They agree, which shows that comparing many configurations with CV did not fool us (picking the best of many can pick a lucky one, which would make the test score worse than the CV score).

“**Evaluation**” means *how* we score: which metrics, on which data, broken down how. “Testing” is the data; “evaluation” is the measuring.

Cheat sheet: data preparation

- **Leakage rule:** after the split, fit everything (MI, percentiles, μ , σ) on training rows only.
- **Cleaning:** removed 1,711 exact duplicates *before* splitting $\rightarrow$ 44,019 rows.
- **Relevance:** Pearson (linear), Spearman (monotonic), MI (any), plus a noise column and a tree leave-one-out check. None uninformative. $\mathbf{F2} = \mathbf{F1} \times \mathbf{F3}$ dropped as redundant. F5 kept after an MLP check (5/5 seeds). F7: $r \approx 0$ but highest MI. 8 inputs.
- **Outliers:** IQR/z-score rules flag the natural tail of skewed data. $\log_{1p}$, then clip to the training 0.5–99.5th percentile. No rows deleted. Target never clipped.
- **Scaling:** z-score (zero-centred, matches Xavier/He, keeps sigmoid/tanh responsive, robust to F7’s extremes). Target standardised, metrics reported in Å.
- **Splitting:** 80/20 stratified on 3 Å bins (seed 42); 5-fold stratified CV on train; 10% of each fold’s training part for early stopping. Limitation: no protein IDs.
- **Imbalance:** 34% vs 7% bins. Stratified folds + stratified test + per-bin RMSE and bias. Loss weighting rejected (4.09 $\rightarrow$ 4.31 Å; high-RMSD bias barely moved).
- **Metrics:** RMSE (headline), MAE, R^2 . CV score chooses; test score reports, once.

2.9 Likely questions: data preparation

Q1. How did you decide whether a feature was uninformative? Why is correlation alone not enough?

We compared each feature with the target using Pearson (linear), Spearman (monotonic) and mutual information (any relationship), with a random-noise column as the “useless” reference, and checked with a model that left out one feature at a time. Correlation only sees straight-line or monotonic patterns. F7 had $r = -0.003$ but the highest MI of all features, so a correlation filter would have removed our most informative feature. No feature was uninformative.

Q2. Did you remove any features? Justify it.

Only F2, because it is *redundant*, not uninformative: $F2 = F1 \times F3$ exactly, so it carries no new information and adding it back changed nothing. F5 also looked redundant ($F5 \approx 138.5 \times F1$), but the MLP was consistently better with it (5/5 seeds), because the ratio $F5/F1$ carries information. Redundancy depends on the model.

Q3. How did you identify outliers, and how did you handle them? Why not just delete them?

IQR and z-score counts, before and after a log transform. Most “outliers” were the natural tail of skewed measurements and vanished after $\log_{1p}$; only F7 had genuine anomalies, and those rows had ordinary targets. We log-transformed the skewed features and clipped every feature to its training 0.5th–99.5th percentile. Deleting would throw away valid targets, would be impossible at prediction time, and would make the test set easier than reality.

Q4. Why should an outlier in the target not be removed?

A large target value (a very bad decoy, a rare class, a high temperature) is real and is often exactly what we most need to predict. Removing it teaches the model that such values do not exist and makes the evaluation dishonest.

Q5. Which scaling did you use and why, given your feature types?

All features are continuous (F8 is a count with hundreds of levels), so one treatment fits all: log1p for skewed features, clip, then z-score standardisation. Z-scores are zero-centred (avoids all of a neuron's gradients sharing one sign), match the unit-variance assumption of Xavier/He initialisation, keep sigmoid/tanh out of their flat regions, and unlike min-max they are not dominated by one extreme value. Categorical features would need one-hot encoding; we had none.

Q6. Why must the scaler be fitted on the training data only?

To prevent data leakage. If test rows influenced the mean, standard deviation or clip limits, information about the test set would be built into the model, and the test score would no longer be an honest estimate. It also mirrors real use: future data is not available when you fit.

Q7. Describe your split strategy and justify it.

80/20 train/test, stratified on 3 Å RMSD bins with a fixed seed; the test set is used once at the end. All comparisons use 5-fold stratified CV on the training set (the spec requires k-fold). Inside each fold, 10% of the training part is held out for early stopping. 20% of 44k rows gives a precise test estimate, each validation fold still has about 7k rows, and 5 folds keep the many experiments affordable.

Q8. What is k-fold cross-validation and why use it instead of a single validation set or the test set?

See the k-fold section above. Short version: the training data is split into k folds; each fold takes a turn as the scored fold while the model trains on the rest; you report mean $\pm$ std. It gives a more reliable score than one split (less luck), shows the noise level, uses all the data, and keeps the test set untouched so that it stays a fair final exam.

Q9. Why stratify a split, and how can you stratify a regression target?

So every part has the same target distribution, especially enough of the rare cases. For regression, bin the continuous target into ranges and stratify on the bin labels.

Q10. Why is a separate early-stopping set needed inside each fold?

Choosing the stopping epoch is a decision. Making it with the fold we report would let that fold influence the model, so the reported score would be optimistic. A separate 10% keeps “choosing” and “scoring” apart.

Q11. How did you handle imbalance during training, testing and evaluation?

Training: stratified folds and early-stopping splits (and we tested inverse-frequency loss weights, which we rejected). Testing: a stratified test split. Evaluation: per-bin RMSE and bias next to the overall metrics. For a classification dataset the equivalents would be stratified splits, class weights or resampling, and per-class metrics such as precision, recall and F1 instead of plain accuracy.

Q12. Why can a single overall metric be misleading on imbalanced data?

It is dominated by the common cases. A model can look good overall while failing badly on the rare ones. In classification, predicting “not a pulsar” for everything gives 91% accuracy on HTRU2 and is useless. For us, overall RMSE 3.78 Å hides a 6.56 Å error on the worst decoys.

Q13. Why did reweighting the loss not help?

It traded accuracy on the common bins for accuracy on the rare bins (overall RMSE 4.09 $\rightarrow$ 4.31 Å),

and it did not fix the under-prediction of bad decoys, because that comes from the features not separating those decoys, not from the loss ignoring them.

Q14. Why remove duplicates before splitting rather than after?

Once the split is made, copies are already on both sides; the model would be scored on rows it memorised (leakage).

Q15. Why might your test score still be optimistic?

There are no protein IDs, so similar decoys of the same protein can appear in both train and test. We could not group the split by protein. Comparisons between settings are unaffected because all settings use the same splits.

3 MLP building blocks (refresher)

3.1 Our network

Input (8) $\rightarrow$ [Linear $\rightarrow$ activation] repeated for each hidden layer $\rightarrow$ Linear $\rightarrow$ output (1 number).

- The **output layer has no activation**, because this is regression and the output must be free to take any value (the standardised target is often negative). A sigmoid would cap it; a ReLU would forbid negatives.
- **Parameter count** of a layer = inputs $\times$ outputs + outputs (biases). The 2×64 baseline: $(8 \cdot 64 + 64) + (64 \cdot 64 + 64) + (64 \cdot 1 + 1) = 4,801$. The 4×256 network has about 200,000.
- Built in PyTorch, but the training loop, LR schedules, pruning masks and k-fold loop are written by hand in `src/`, so every step can be explained.

3.2 One training step

For every mini-batch of 256 rows (`src/training.py`):

1. **Set the learning rate** for this step from the schedule. This is where warmup happens.
2. **Forward pass**: each layer computes activation($Wx + b$), producing predictions.
3. **Loss**: mean squared error between predictions and the true (standardised) targets.
4. **Backward pass (backpropagation)**: the chain rule, applied from the loss back through every layer, gives the gradient $\partial \text{loss} / \partial w$ for every weight. (Gradients are cleared first, because PyTorch adds new gradients to old ones.)
5. **Update**: the optimiser moves every weight against its gradient. Plain SGD: $w \leftarrow w - \eta g$.
6. (Part 2 only) re-apply the pruning masks so pruned weights stay at zero.

One **epoch** = one pass over all training rows (≈ 100 steps of 256 rows). The rows are reshuffled every epoch. A **mini-batch** gradient is a noisy estimate of the full gradient; the noise is cheap and even helps escape poor regions.

3.3 Learning rate and schedules

The **learning rate** η is the size of each step. Too small: slow. Too large: the step overshoots the valley and the loss can explode (**divergence**). A **schedule** changes η during training:

- **Constant**: the same η throughout.
- **Cosine decay**: η follows half a cosine wave from its peak down to 0. Large steps early for progress, small steps late so the weights settle into a minimum instead of bouncing around it.
- **Linear warmup**: η rises in a straight line from nearly 0 to the peak over the first few epochs, then the normal schedule continues. Part 2 uses 5 epochs of warmup followed by cosine decay.

3.4 Weight initialisation

Goal: keep the size (variance) of the signals roughly the same from layer to layer, so they neither explode nor vanish, in both the forward and the backward pass.

- **Xavier / Glorot**: variance $2/(\text{fan_in} + \text{fan_out})$. Designed for symmetric activations (sigmoid, tanh).
- **He / Kaiming**: variance $2/\text{fan_in}$, roughly twice Xavier's, because ReLU sets about half its inputs to zero and so halves the variance. Used with ReLU and its variants.

- **Naive random** (e.g. a fixed standard deviation of 0.01): ignores layer size, so in deep networks the signal shrinks layer after layer and training stalls.
- Biases start at 0, but weights must **not** all start equal: then every unit in a layer computes the same thing and gets the same update forever (no **symmetry breaking**).

We always matched the initialisation to the activation (“auto”: Xavier for sigmoid/tanh, He for ReLU).

3.5 Overfitting and early stopping

Overfitting: the training error keeps falling, but the error on unseen data stops improving and eventually rises, because the network starts memorising details of its training rows. Our baseline learning curves show it: training MSE keeps dropping while the early-stopping MSE flattens after about 100 epochs.

Early stopping measures the error on the separate “stop” rows after every epoch, remembers the best epoch’s weights, and stops after 20 epochs without improvement (**patience** 20, because the curve is noisy). It then **restores the best weights**, not the last ones. It is a form of **regularisation**: it limits how far the network can overfit. (Maximum 500 epochs.)

3.6 Regularisation in general (the axis we did not test)

Regularisation = anything that stops a model from overfitting. The spec lists three:

- **Weight decay (L2):** add $\lambda \sum w^2$ to the loss, so every step also shrinks the weights a little. Prefers simple, small-weight solutions. (It would interact with magnitude pruning, because it changes which weights are small.)
- **Dropout:** during training, randomly switch off a fraction of units at each step, so the network cannot rely on any single unit; like training many thinned networks at once. Switched off at evaluation.
- **Early stopping:** stop when the validation error stops improving (we used this).

Expected behaviour: without regularisation, a large network’s training/validation gap grows; with it, the gap shrinks, sometimes at a small cost in training fit. Too much regularisation causes underfitting.

3.7 Baselines and noise

Why baselines? “RMSE 4.09 Å” means nothing on its own. Predict-the-mean (6.14 Å, $R^2 = 0$) is the floor any model must beat. Linear regression (5.19 Å, $R^2 = 0.29$) tests whether non-linearity is needed (it is: the MLP is ~21% better). Gradient-boosted trees (4.13 Å) are a strong non-neural reference; matching them shows our MLP is trained sensibly. All three are ready-made scikit-learn models, run through our own CV code with the same folds and preprocessing (notebook §2.2).

Two kinds of noise: *fold-to-fold* (std 0.03–0.05 Å, from which rows are in each fold) and *seed-to-seed* (~0.015 Å, from the random initial weights and batch order). A **paired comparison** runs both settings with the same seeds and folds and looks at the per-seed *difference*, so shared noise cancels out.

Cheat sheet: MLP building blocks

- Training step: set LR → forward → MSE loss → backprop → optimiser step (+ masks in Part 2).
- Output layer has no activation (regression). Parameters = in × out + out per layer.
- Xavier $2/(\text{in} + \text{out})$ for sigmoid/tanh; He $2/\text{in}$ for ReLU. Never all-equal weights.

- Early stopping: watch a separate split, keep the best epoch, patience 20. It is regularisation.
- Regularisation family: weight decay (shrink weights), dropout (random units off), early stopping.
- Schedules: constant, cosine decay, linear warmup then decay.
- Baselines: mean 6.14, linear 5.19, trees 4.13, MLP 2×64 4.09 Å.
- Noise: seed ~ 0.015 Å, fold std 0.03–0.05 Å. Use paired comparisons.

3.8 Likely questions: building blocks

Q16. Walk through one training step of an MLP.

Set this step's learning rate; forward pass to get predictions; compute the loss; zero the old gradients; backpropagate to get every weight's gradient; let the optimiser update the weights.

Q17. Why compare against trivial baselines?

They give the error a scale. Without them, nobody can tell whether 4.09 Å is good. Predict-the-mean is the floor; linear regression shows whether a non-linear model is needed.

Q18. What does early stopping monitor and why does it help?

The error on a held-out split after each epoch. It keeps the best weights and stops once the error has not improved for a while, which prevents the network from carrying on into overfitting.

Q19. Why pair Xavier with tanh/sigmoid and He with ReLU?

Both keep the signal variance stable across layers. ReLU discards about half of its inputs, so He uses twice the variance to compensate.

Q20. The training loss keeps falling but the validation loss flattens and rises. What is happening, and what can you do?

Overfitting. Use early stopping, weight decay, dropout, more data, or a smaller network.

Q21. How do you know whether a difference between two settings is real?

Compare it with the noise: fold-to-fold and seed-to-seed spread. We required a difference larger than that noise *and* in the same direction in every seed (a paired comparison).

4 Core investigation (the axes)

4.1 What we did and how

The spec asks for a baseline and controlled experiments on at least two of five axes (initialisation, activation functions, architecture, optimisers, regularisation), evaluated with k-fold CV, with a hypothesis stated before each one. We chose **three**:

- **Optimisers**, because warmup in Part 2 is a learning-rate question, and optimisers decide how the LR acts.
- **Architecture**, to choose a principled network size, with enough spare capacity to prune.
- **Activation functions**, chosen instead of regularisation because they are easy to reason about, and they let us show vanishing gradients directly. Initialisation was always matched to the activation.

Protocol for every axis. Hypotheses written and committed before running. Change one thing; everything else stays at the baseline (2×64, ReLU, He, Adam 10^{-3} , batch 256, early stopping). Each configuration: **3 seeds** × **5 folds**. Same four measurements on every axis:

Final quality	CV RMSE of the early-stopped model (mean over folds, averaged over seeds)
Convergence speed	Epochs until the early-stopping MSE first drops below 0.50 (standardised units: predicting the mean scores 1.0, the baseline ends at about 0.43)
Stability	Folds that diverged, and the spread across folds and seeds
Learning curves	Early-stopping MSE per epoch, to see <i>how</i> configurations differ

The mistake that framed our hypotheses (worth mentioning)

In the baseline phase, the MLP and gradient-boosted trees both stopped at $R^2 \approx 0.55$. We concluded that the features set a “ceiling”, and several hypotheses predicted only *small* differences in final quality. The architecture axis then showed a 4×256 MLP reaching $R^2 = 0.615$. The ceiling was really the baseline’s limited capacity. Lesson: two models agreeing is weak evidence of a ceiling; test it by making the model bigger. Being wrong for a clear, identifiable reason is a legitimate result.

4.2 Axis A: Optimisers

The theory

Notation: w = a weight, g = its gradient this step, η = learning rate.

Optimiser	Update	Idea
SGD	$w \leftarrow w - \eta g$	Step straight downhill
SGD + momentum	$v \leftarrow \beta v + g; \quad w \leftarrow w - \eta v$ ($\beta = 0.9$)	Keep a running velocity: steady directions build up speed, zig-zags cancel out
RMSprop	$s \leftarrow \rho s + (1 - \rho)g^2; \quad w \leftarrow w - \eta g / (\sqrt{s} + \epsilon)$	Divide each weight’s step by its recent typical gradient size, so all weights take similar-sized steps
Adam	momentum average m and squared average s , both bias-corrected; $w \leftarrow w - \eta \hat{m} / (\sqrt{\hat{s}} + \epsilon)$	Momentum plus RMSprop-style scaling

- **Momentum’s effective step:** if the gradient stays similar, v grows to $g/(1 - \beta) = 10g$, so the step is about **10× the LR**. Momentum therefore blows up at a lower LR than plain SGD.

- **Adaptive methods** (RMSprop, Adam) make the step size roughly independent of the gradient’s scale, which makes them forgiving of the LR choice and partly hides tiny (vanishing) gradients.
- **Bias correction (Adam only):** m and s start at 0, so early on they underestimate the true averages. Adam divides by $(1 - \beta^t)$ to fix this. PyTorch’s RMSprop has *no* bias correction.

Design

All four optimisers on the **same LR grid** $\{10^{-4}, 3 \cdot 10^{-4}, 10^{-3}, 3 \cdot 10^{-3}, 10^{-2}, 3 \cdot 10^{-2}, 10^{-1}\} \times 3$ seeds. One fixed LR would be unfair because each optimiser has its own natural LR scale; the grid finds each one’s best LR *and* shows how sensitive it is to the choice.

Hypotheses, results and verdicts

Optimiser	Best LR	CV RMSE	Epochs to 0.50	Good LR ^s *
Adam	10^{-2}	3.99	9.7	3
SGD + momentum	$3 \cdot 10^{-2}$	4.03	12.5	2
RMSprop	$3 \cdot 10^{-3}$	4.21	29.5	0
SGD	$3 \cdot 10^{-3}$	4.39	293 (6/15 folds)	0

* grid LR^s giving a model within 0.10 Å of the overall best.

H-A1 (speed): “Adam and RMSprop fastest, plain SGD slowest, momentum in between.”

Partly supported Adam was fastest and SGD slowest, as predicted. But **momentum beat RMSprop**: momentum alone gave about a 20× speed-up over plain SGD. *Why we were partly wrong*: we assumed the adaptive scaling was the main source of speed. In fact much of Adam’s speed comes from the momentum it contains.

H-A2 (final quality): “At their best LR^s, all four end within 0.05 Å.” **Contradicted** The spread was 0.40 Å. *Why*: with a fixed budget (500 epochs) and early stopping, plain SGD never reached the good region. At small LR^s it hit the 500-epoch limit still improving; at larger LR^s early stopping ended it early at a worse score. **Under a limited budget, speed becomes final quality.** The hypothesis also relied on the false “ceiling”.

H-A3 (LR tolerance): “Adam and RMSprop are good over a wider LR range than SGD; momentum diverges at a lower LR than plain SGD.” **Mixed** Adam was the most tolerant (3 good LR^s), as predicted. **RMSprop was the least tolerant**: it diverged in 11/15 folds at $3 \cdot 10^{-2}$ and all folds at 10^{-1} . The momentum part held: momentum broke at 10^{-1} while plain SGD still trained there, consistent with its $\sim 10\times$ effective step. *Why RMSprop failed (we measured it)*: without bias correction, its squared-gradient average starts at 0, so its **first update is 10× the LR**, while Adam’s is exactly 1×. Oversized steps at the very start of training are exactly the problem **warmup** solves, which links this axis to Part 2.

Side effect: dead units. Large LR^s knock ReLU units into permanently “off”: Adam’s dead fraction rose from 0.3% (10^{-3}) to 10%, 32% and 48% (10^{-1}).

4.3 Axis B: Architecture (depth vs width)

The theory

- **Width** = units per layer: how many features a layer computes side by side.
- **Depth** = number of layers: how many times features can be built *from* other features (composition).
- **Universal approximation**: one hidden layer can approximate any continuous function given enough units, but depth can often do the same with far fewer units.

- **Costs:** depth makes the gradient path longer (risk of vanishing/exploding gradients, harder optimisation). Size in general means more capacity to memorise (overfitting) and slower training.

Design

Depth $\{1, 2, 3, 4\} \times$ width $\{16, 64, 256\}$: 12 networks from 161 to ~ 200 k parameters, $\times 3$ seeds, baseline otherwise. Also recorded: parameter count, best epoch, and the train-validation gap at the kept epoch (an overfitting indicator). Limitation: every size used the same LR (10^{-3}).

Results (CV RMSE in Å, mean of 3 seeds; seed std ≤ 0.03)

depth \ width	16	64	256
1	4.63	4.39	4.23
2	4.42	4.09 (baseline)	3.95
3	4.33	4.00	3.83
4	4.30	3.97	3.81 (R^2 0.615)

H-B1 (width): “16 $\rightarrow$ 64 clearly helps; 64 $\rightarrow$ 256 helps less than 0.05 Å.” Partly supported

16 $\rightarrow$ 64: -0.33 Å; 64 $\rightarrow$ 256: -0.14 Å. Returns do diminish, but the second gain is about $3\times$ what we predicted. *Why wrong:* we assumed the feature ceiling; the baseline was capacity-limited.

H-B2 (depth): “2 layers beat 1; beyond 2, less than 0.05 Å gain.” Half supported 1 $\rightarrow$ 2

layers: -0.30 Å, as predicted. But 2 $\rightarrow$ 3 (-0.09) and 2 $\rightarrow$ 4 (-0.12 Å) still helped clearly. *Why wrong:* same ceiling assumption.

H-B3 (overfitting): “Bigger nets overfit sooner, but early stopping stops them ending worse.” Supported The kept epoch fell from ~ 310 (1×16) to ~ 54 (4×256), and the train-validation gap rose from ≈ 0 to 0.19, so bigger nets overfit sooner and more. Early stopping caught them in time, and the biggest net finished best.

Extra findings: depth was more parameter-efficient than width: 4×64 (13k parameters) matched 2×256 (68k) to within 0.014 Å. Bigger nets also learned faster: 4×256 reached $\text{MSE} < 0.50$ in 5 epochs vs 21 for 2×64 .

4.4 Axis C: Activation functions

The theory

Activation	Formula	Max derivative	Notes
Sigmoid	$1/(1 + e^{-x})$	0.25	Output (0, 1): not zero-centred. Saturates (flat) for large $ x $
tanh	$(e^x - e^{-x})/(e^x + e^{-x})$	1	Output $(-1, 1)$: zero-centred. Still saturates
ReLU	$\max(0, x)$	1 (0 if $x < 0$)	No saturation for $x > 0$, cheap. Units can “die”
Leaky ReLU	x if $x > 0$, else $0.01x$	1 (0.01 if $x < 0$)	Never completely dead

- **Vanishing gradients:** backprop multiplies by one activation derivative per layer. For sigmoid that factor is at most 0.25, so after L layers the gradient reaching the first layer can shrink by up to 0.25^L . Early layers then barely learn. ReLU’s derivative is exactly 1 for active units, so gradients pass through undiminished.

- **Zero-centring:** sigmoid outputs are all positive, so the next layer’s inputs are all positive and its weight gradients share one sign (zig-zag updates). tanh avoids this.
- **Dead ReLU:** if a unit’s input is ≤ 0 for every row (e.g. after a huge update), it outputs 0, gets zero gradient and can never recover. Leaky ReLU keeps a small slope so it can come back.

Design

Sigmoid, tanh, ReLU and Leaky ReLU (with matching initialisation), at 2×64 and a deeper 4×64 , $\times 3$ seeds, Adam 10^{-3} . Two diagnostics: (1) **gradient flow at initialisation:** the average gradient size in the first layer divided by that in the last layer; far below 1 means vanishing gradients. (2) **dead units:** the fraction of ReLU units that output 0 for every validation row.

Results

Activation	RMSE 2×64	RMSE 4×64	Epochs to 0.50 (2×64 / 4×64)
Sigmoid	4.34	4.26	423 / 340
tanh	4.06	3.90	64 / 21
ReLU	4.09	3.97	21 / 8.6
Leaky ReLU	4.10	3.96	21 / 8.5

Gradient flow at initialisation (first layer’s gradient size $\div$ last layer’s; far below 1 = vanishing)

	First $\div$ last	depth 2	depth 4	depth 8
Sigmoid		2.7e-3	1.7e-4	5.8e-7
tanh		0.31	0.27	0.33
ReLU		0.095	0.091	0.080

H-C1 (speed): “ReLU family fastest, then tanh, then sigmoid.” Supported 8.6 / 21 / 340 epochs at 4×64 . Sigmoid’s small derivative and non-zero-centred output make it slow.

H-C2 (vanishing gradients): “Sigmoid’s gradient ratio is far smaller and shrinks with depth; its RMSE gap grows with depth, but Adam softens the effect.” Supported With sigmoid the gradient shrinks by **about $4 \times$ per layer** going backwards, exactly what its maximum derivative of 0.25 predicts; with 8 layers the first layer gets about a millionth of the output layer’s gradient. tanh and ReLU stay flat across depths. But in RMSE the sigmoid gap grew only slightly ($0.25 \rightarrow 0.30 \text{ \AA}$), and sigmoid even improved from depth 2 to 4. *Why:* Adam divides each weight’s step by that weight’s own gradient size, so tiny gradients still give normal-sized steps. The damage showed up as *slow* training, not failure. With plain SGD it would have been far worse.

H-C3 (ReLU vs Leaky ReLU): “tanh, ReLU and Leaky ReLU within $0.05 \text{ \AA}$; Leaky gains nothing because few ReLU units die.” Partly supported Leaky ReLU = ReLU, and fewer than 0.5% of ReLU units died, so the reasoning held: there was nothing for Leaky to fix. But **tanh beat ReLU at depth 4** (3.90 vs 3.97 $\text{\AA}$, 3/3 seeds) while training $2.5 \times$ slower. A plausible but untested reason: tanh networks are smooth, while ReLU networks are piecewise linear.

4.5 The axes we did not run (in case they are asked)

Initialisation (random vs Xavier/He). *Hypothesis we would make:* a fixed small-random initialisation (std 0.01) trains much more slowly, and fails in deeper nets, because the signal shrinks layer by layer; Xavier suits tanh/sigmoid, He suits ReLU. *How to test:* same network and optimiser, swap only the initialisation, compare convergence speed and the gradient-flow ratio at several depths. Our code supports `init="random"`.

Regularisation (weight decay, dropout, early stopping). *Hypothesis we would make:* on our large 4×256 network, weight decay or dropout would reduce the train–validation gap and allow later stopping, with at most a small gain in CV RMSE, because early stopping already controls overfitting well. *How to test:* a grid of decay strengths and dropout rates, measure the gap and the CV RMSE.

4.6 Final test evaluation

After all CV comparisons, the test set was used **once** (notebook §3.5): the best CV-validated configuration (4×256 , ReLU, Adam 10^{-3}) and the baseline, each trained on the whole training set (90% fit, 10% stop) with 5 seeds. We deliberately did *not* try untested combinations (such as Adam at 10^{-2} with 4×256 , or tanh), because picking between them on the test set would be selecting on the test set.

Model	CV RMSE	Test RMSE	Test R^2
Predict the mean	6.14	6.14	0.00
Linear regression	5.19	5.19	0.29
Baseline 2×64	4.09	4.05 ± 0.04	0.566
Best 4×256	3.81	3.78 ± 0.03	0.621

CV and test agree, so model selection did not overfit the validation folds. The bigger network helped most at the extremes ($0\text{--}3 \text{ \AA}$: $3.08 \rightarrow 2.75 \text{ \AA}$; $18\text{--}21 \text{ \AA}$: $7.09 \rightarrow 6.56 \text{ \AA}$), reducing but not removing the regression to the mean.

4.7 Choosing the Part 2 network

4×256	Best RMSE, and plenty of spare capacity for pruning to remove
ReLU, He	Fastest to train, and the standard in the pruning papers we follow
SGD + momentum 0.9	It has a sharp LR-tolerance boundary (fine at $3 \cdot 10^{-2}$, broken at 10^{-1}), which is exactly what the warmup question is about. Adam degrades gradually and self-corrects its early steps (bias correction), which would blur the effect. The original papers also used SGD with momentum

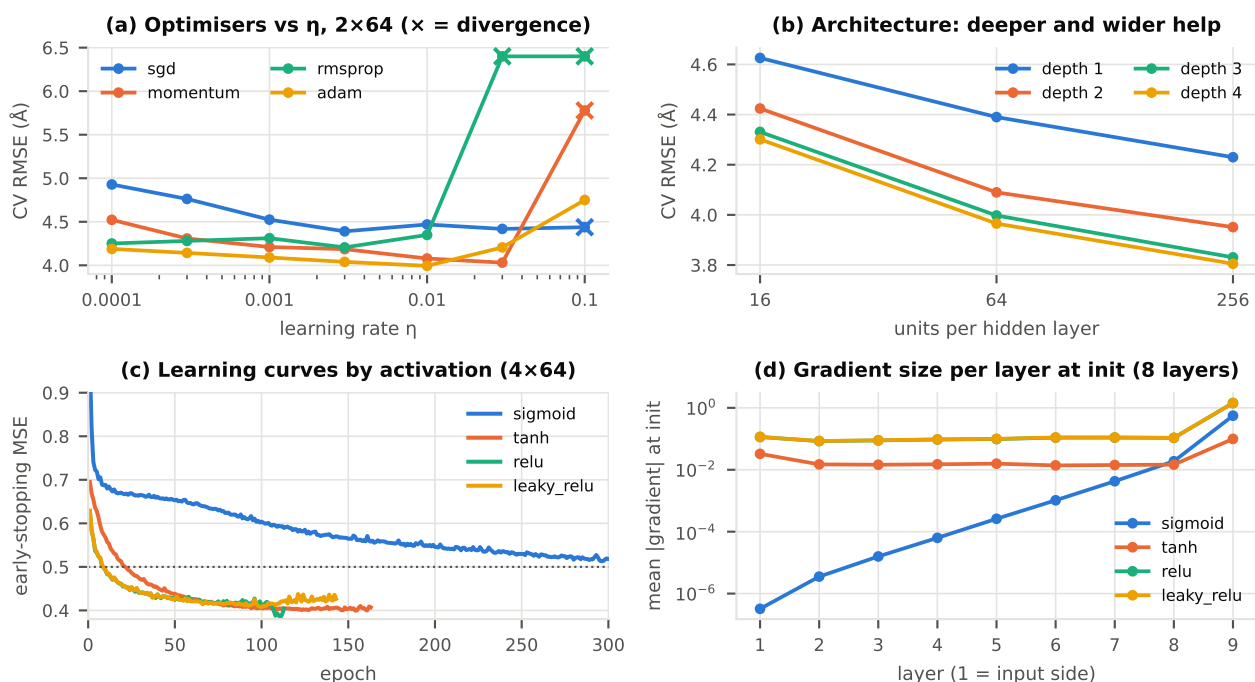


Figure 2: Core investigation. (a) Each optimiser across the LR grid (× = divergence). (b) RMSE by depth and width. (c) Learning curves by activation. (d) Gradient size per layer at initialisation, 8 hidden layers: sigmoid’s collapses towards the input.

Cheat sheet: core investigation verdicts

Hypothesis	Verdict	Key evidence and reason
A1 Adam/RMSprop fastest, SGD slowest	Partly	Adam 9.7, momentum 12.5, RMSprop 29.5, SGD 293 epochs. Momentum is the main speed source
A2 All optimisers end within 0.05 Å	Contradicted	3.99–4.39 Å. Under a fixed budget, speed becomes quality
A3 Adaptive = LR-tolerant; momentum breaks before SGD	Mixed	Adam most tolerant; RMSprop least (no bias correction, first step 10× LR); momentum part true (10× effective step)
B1 Width: 64 → 256 adds <0.05 Å	Partly	−0.33 then −0.14 Å
B2 Depth: beyond 2 layers adds <0.05 Å	Half	1 → 2: −0.30; 2 → 4: −0.12 Å
B3 Big nets overfit sooner; early stopping saves them	Supported	Best epoch 310 → 54; gap 0 → 0.19; 4×256 best (3.81 Å)
C1 Speed: ReLU > tanh > sigmoid	Supported	8.6 / 21 / 340 epochs
C2 Sigmoid gradients vanish with depth; Adam softens it	Supported	≈4× shrink per layer (0.25 derivative); shows in speed, not final RMSE
C3 tanh ≈ ReLU ≈ Leaky	Partly	Leaky = ReLU (<0.5% dead); tanh beat ReLU at depth 4

Big lesson: several hypotheses failed because of the false “feature ceiling”; the baseline was capacity-limited. Best model: 4×256, CV 3.81 Å, test 3.78 Å, R^2 0.621.

4.8 Likely questions: core investigation

Q22. Which axes did you investigate and why those?

Optimisers (they control how the LR acts, which feeds the warmup question), architecture (to choose a principled network size with capacity to prune) and activation functions (easy to reason about, and

they show vanishing gradients).

Q23. State a hypothesis for one axis and say whether your results supported it.

Use the template: e.g. H-C2: “sigmoid’s gradients shrink with depth because its derivative is at most 0.25.” Design: gradient ratio first/last layer at depths 2, 4, 8. Result: $\approx 4\times$ shrink per layer, ratio $5.8e-7$ at depth 8; tanh and ReLU flat. Verdict: supported. Why: the chain rule multiplies one derivative per layer. Nuance: Adam hid most of the effect in final RMSE.

Q24. Why did you compare optimisers on a grid of learning rates rather than at one learning rate?

Each optimiser has its own natural LR scale (Adam around 10^{-3} , SGD higher). At one LR you mostly measure which optimiser that LR happens to suit. The grid compares each at its best LR and shows its sensitivity.

Q25. Explain how momentum speeds up SGD and why it can be less stable.

It accumulates a velocity: directions the gradient keeps pointing in build up speed, and oscillating directions cancel. With $\beta = 0.9$ the velocity reaches about $10\times$ the gradient, so the effective step is about $10\times$ the LR; at a large LR that overshoots, which is why momentum diverged at 10^{-1} while plain SGD did not.

Q26. What does Adam adapt, and why is it forgiving of the learning rate?

It keeps a momentum average and a squared-gradient average per weight and divides the step by the square root of the latter. Step sizes become roughly independent of gradient scale, so a wide range of LRs works. Bias correction stops the first steps from being oversized.

Q27. How did you measure convergence speed and stability?

Speed: epochs until the early-stopping MSE first falls below 0.50. Stability: number of diverged folds and the spread across folds and seeds.

Q28. Depth vs width: what does each give you, and which was better here?

Width gives more features per layer; depth lets features be built from features. Depth costs longer gradient paths. Here depth was more parameter-efficient (4×64 with 13k parameters matched 2×256 with 68k).

Q29. Bigger networks overfit more. Why was the biggest one best?

Early stopping halted it at about epoch 54, before overfitting hurt the validation error. Capacity helps as long as something controls the overfitting.

Q30. Why does sigmoid cause vanishing gradients, and why do tanh and ReLU help?

Its derivative is at most 0.25 and near 0 when saturated; backprop multiplies these per layer, so gradients shrink exponentially with depth. tanh’s derivative reaches 1 and it is zero-centred; ReLU’s derivative is exactly 1 for active units.

Q31. What is a dead ReLU, and did it matter for you?

A unit whose input is negative for every example outputs 0 and gets no gradient, so it never recovers. At normal LRs fewer than 0.5% died, so Leaky ReLU gave no benefit; at Adam LR 10^{-1} , 48% died.

Q32. One of your hypotheses was contradicted. Explain why.

H-A2 (all optimisers end equal): under a fixed epoch budget and early stopping, the slow optimiser never reached the good region, so speed turned into quality. Also it relied on a feature ceiling that turned out to be the baseline’s capacity.

Q33. Why did you use only CV-validated configurations on the test set?

Trying new combinations on the test set and keeping the best would make the test score optimistic. The CV and test scores agreeing (3.81 vs 3.78 Å) shows selection did not overfit.

5 Research investigation: warmup and pruning

5.1 The question, in plain words

The literature reports that a network trained with **learning-rate warmup** survives **pruning** better than one trained without it, even when both are equally accurate before pruning. The proposed explanation is a two-link chain:

warmup $\xrightarrow{\text{link 1, [3]}}$ a larger LR becomes trainable $\xrightarrow{\text{link 2, [2]}}$ better recovery from pruning

So warmup might improve pruning robustness *indirectly*. The spec stresses this is a hypothesis to test, not a fact.

5.2 The three papers

- [1] **Frankle & Carbin (2019), the Lottery Ticket Hypothesis**. A dense network contains a sparse subnetwork (a “winning ticket”, often 10–20% of the weights) that, trained from the *same initial weights*, matches the full network. Found by iterative magnitude pruning. For deeper networks, warmup and a constant LR gave similar unpruned accuracy, but warmup gave much higher accuracy as the network was pruned further (Appendix I.5).
- [2] **Renda, Frankle & Carbin (2020), rewinding vs fine-tuning**. Three ways to retrain after pruning: **fine-tuning** (continue from the final weights at a small fixed LR), **weight rewinding** (reset the surviving weights to an earlier point and retrain with the original schedule) and **LR rewinding** (keep the final weights, retrain with the original, large-LR schedule). Both rewinding methods beat fine-tuning, so a *larger retraining LR helps recovery*.
- [3] **Kalra & Barkeshli (2024), why warmup?** Warmup’s main benefit is letting the network **tolerate a larger target LR**. At the start the loss surface can be sharp; small early steps move the network into a flatter, better-conditioned region, where large steps are safe.

5.3 Key concepts

- **Warmup**: start with a tiny LR and raise it (here linearly over 5 epochs) to the target “peak” LR, then follow the normal schedule (here cosine decay to 0).
- **Pruning**: setting some weights to exactly zero to make the network smaller and faster. **Sparsity** = the fraction of weights removed (70% sparsity keeps 30%).
- **Magnitude pruning**: remove the weights with the smallest absolute value $|w|$, assuming they matter least.
- **Global vs layer-wise**: *global* ranks all weights of all layers together, so layers can end up with different sparsities; *layer-wise* removes the same fraction from every layer. We used **global**, and never pruned biases (few, and each shifts a whole unit).
- **One-shot vs iterative**: one-shot prunes to the target in one go; iterative prunes a little, retrains, repeats (as in [1]). We used **one-shot** (simpler, allowed by the spec, possibly a weaker effect).
- **Masks**: a 0/1 grid per weight matrix. During fine-tuning the masks are re-applied after every optimiser step, because the gradient and momentum would otherwise push pruned weights away from zero.
- **Robustness** = how much accuracy the network keeps when damaged by pruning. We measure it with Δ .

Δ : our robustness measure

$\Delta(s)$ = test RMSE after pruning $s\%$ and fine-tuning – test RMSE of the unpruned network. Computed per seed, then averaged. **Smaller Δ = more robust.** Using a difference removes the effect of slightly different starting accuracies, so it isolates the damage done by pruning.

5.4 Fixed setup

Network	4×256, ReLU, He ($\sim 200k$ weights)
Optimiser	SGD + momentum 0.9, batch 256
Budget	Fixed 100 epochs, no early stopping (final weights kept), so every condition trains equally long
Schedule	Cosine decay to 0. Warmup conditions add 5 epochs of linear warmup first. <i>The only difference is the warmup</i>
Data	Training set split once into fit 90% / validation 10%. All choices on validation; pruning results reported on the test set
Seeds	3 for the LR sweep, 5 for the conditions (spec: ≥ 3). Mean $\pm$ std
Pruning	One-shot global magnitude pruning at 0, 30, 50, 70, 90, 95, 98%
Fine-tuning	Identical for all: SGD + momentum, constant LR 0.001, 10 epochs, masks re-applied

5.5 Step 1: does warmup raise the tolerable learning rate? (H1)

H1: “The maximum stable LR with warmup is at least one grid step higher than without.” *Max stable LR* = the largest grid LR where all 3 seeds train without diverging and the validation RMSE is within $0.10 \text{ \AA}$ of that schedule’s best.

Design: train with and without warmup on the LR grid $\{0.005, 0.01, 0.02, 0.03, 0.05, 0.1, 0.2\} \times 3$ seeds; record validation RMSE and divergence.

Peak LR	0.005	0.01	0.02	0.03	0.05	0.1	0.2
No warmup (diverged seeds)	3.73 (0)	3.80 (2/3)	– (3)	– (3)	– (3)	– (3)	– (3)
Warmup (diverged seeds)	3.68 (0)	3.63 (0)	3.63 (0)	3.66 (0)	3.71 (0)	3.71 (0)	3.73 (0)

Strongly supported Max stable LR **0.005 without warmup, 0.1 with warmup** (0.2 also trains but is a hair outside the $0.10 \text{ \AA}$ tolerance): **at least $20\times$ higher**, 4 grid steps. **All 17 no-warmup failures happened in the first epoch.** *Why:* at initialisation the loss surface is sharp, and full-size steps (amplified $\sim 10\times$ by momentum) overshoot immediately. Warmup’s small first steps move the network to a calmer region before the big steps begin, exactly the mechanism in [3]. Warmup also gave better unpruned models (3.63 vs $3.73 \text{ \AA}$).

5.6 Step 2: choosing the conditions (rules fixed in advance)

A: no warmup , η_0	η_0 = the no-warmup LR with the best validation RMSE $\rightarrow$ 0.005
B: warmup , $\eta_1 > \eta_0$	The <i>largest</i> warmup LR above η_0 that is stable and within 0.05 $\text{\AA}$ of A’s validation RMSE (“comparable unpruned accuracy”) $\rightarrow$ 0.2
C: warmup , η_0 (control)	Same LR as A, but with warmup $\rightarrow$ 0.005

Why “comparable unpruned accuracy”? The spec requires it, so that differences after pruning reflect robustness, not a better starting point. Warmup LRs 0.01–0.03 were about $0.1 \text{ \AA}$ *better* than A, so they were not comparable; LR 0.2 matched A (validation 3.73 vs $3.74 \text{ \AA}$ over 5 seeds).

Why the control C? A and B differ in *two* things at once: warmup *and* learning rate. If B is more robust, we could not tell which caused it. C has warmup but A’s small LR, so it separates “warmup itself” from “the larger LR warmup allows”.

Why fix the rules first? Otherwise we could, even unintentionally, pick the B learning rate or the threshold that makes the result look best. The rules were committed to git before running, so the conditions were chosen mechanically.

5.7 Step 3: prune, fine-tune, compare (H2 and H3)

For each condition $\times$ 5 seeds: train 100 epochs; then for each sparsity, prune a copy of the trained network, measure test RMSE (“before fine-tuning”), fine-tune identically, measure again, and compute Δ .

H2: “At comparable unpruned RMSE, B (warmup, large LR) has a smaller Δ than A at high sparsity.”

H3: “The benefit comes from the larger LR, not warmup itself: B beats C, while $C \approx A$.” **Counting rule (fixed in advance):** smaller Δ in $\geq 4/5$ seeds (paired), by more than the standard deviation of the paired differences, at ≥ 2 of the 4 sparsities $\geq 70\%$.

Sparsity	Test RMSE <i>before</i> fine-tuning ($\text{\AA}$)			Δ <i>after</i> fine-tuning ($\text{\AA}$)		
	A	B	C	A	B	C
30%	4.35	3.89	4.60	0.03	0.00	0.03
50%	5.69	4.08	6.15	0.08	0.02	0.11
70%	7.97	4.61	7.39	0.31	0.09	0.35
90%	8.59	5.86	7.82	0.71	0.44	0.74
95%	7.51	6.40	7.15	0.95	0.69	1.01
98%	6.74	6.45	6.69	1.34	1.09	1.40

Unpruned test RMSE: A 3.81, B 3.84, C 3.78 $\text{\AA}$ (so B started slightly *worse*).

Comparison	Mean Δ difference	Seeds	Meets the rule?
B vs A	−0.22 to −0.27 $\text{\AA}$	5/5 at every sparsity	Yes, 4/4 sparsities → H2 supported
B vs C	−0.26 to −0.32 $\text{\AA}$	5/5 at every sparsity	Yes, 4/4
C vs A	+0.03 to +0.06 $\text{\AA}$ (C slightly worse)	C better in 0–2/5	No → with B vs C, H3 supported

H2 Supported B lost 0.22–0.27 $\text{\AA}$ less than A at every sparsity $\geq 70\%$, in 5/5 seeds, about $5\times$ the seed noise. **Before fine-tuning** the gap was dramatic: at 70% sparsity B’s error was 4.6 $\text{\AA}$ while A and C jumped to 8.0 and 7.4 $\text{\AA}$, *worse than predicting the mean* (6.14 $\text{\AA}$). (A and C come back down at 98% only because a nearly empty network outputs almost a constant, which is close to the mean.)

H3 Supported Warmup at the small LR (C) gave *no* benefit over A, even though C had the best unpruned RMSE. Only the large-LR network B was robust. So warmup helps pruning robustness **only through the larger LR it makes possible**.

5.8 Alternative explanations we ruled out (confounders)

- **Dead units as “free” sparsity.** Large LRs can kill ReLU units, and pruning a dead unit’s weights costs nothing. But B had the *fewest* dead units (A 3.6%, B 0.2%, C 0.0%), so this cannot explain it.
- **Fine-tuning LR favouring B.** 0.001 is 20% of A’s and C’s training LR but only 0.5% of B’s. By [2], a relatively larger retraining LR helps recovery, so this setup *favours A and C*. B won anyway.

- **Starting point.** B’s unpruned test RMSE was slightly worse than A’s, so its advantage is not a head start.
- **Reference point.** Measuring Δ against the fine-tuned dense network instead gives the same ranking.

Post-hoc look at the weights (added after seeing the results, so exploration, not a test): A and C have almost identical weight distributions, matching their identical pruning behaviour. B’s weights are larger (median $|w|$ 0.084 vs 0.062) and more of its “weight mass” sits in its largest weights (top 10% hold 60.6% of $\sum w^2$ vs 56.5%). A network that relies more on a few large weights loses less when the many small ones are removed. Another untested explanation: large-LR training ends in flatter minima, where zeroing weights changes the loss less.

5.9 Extensions E1, E2, E3

	Question	Planned in advance?
E1	Does retraining with each network’s own schedule (LR rewinding, from [2]) change the picture?	Yes
E2	Does a longer warmup give more robustness (spec’s “warmup duration” extension)?	Yes
E3	Does robustness rise steadily with the LR among warmup networks (dose–response)?	No, exploratory

E1: LR rewinding. After pruning, retrain for 10 epochs with the network’s *own* schedule squeezed into 10 epochs (same peak LR, warmup for 5% if it had one, cosine to 0), instead of the small fixed LR. Δ is measured against each network’s own retrained dense version, because rewinding improves the unpruned networks too (B 3.84 $\rightarrow$ 3.72 Å). *Prediction:* everyone recovers better, and the A–B gap shrinks. *Result:* everyone recovered better, but the gap **widened** (at 90%: A 0.58 Å, B 0.03 Å). **Gap prediction contradicted** *Why:* B now retrains at LR 0.2 while A retrains at 0.005, and a large retraining LR recovers better ([2]). So the large LR that warmup unlocks helps **twice**: more robust trained weights *and* more effective recovery.

E2: warmup duration (condition B at LR 0.2 with 1, 5 or 15 warmup epochs). *Prediction:* once warmup makes LR 0.2 trainable, longer warmup adds nothing (a threshold effect). *Result:* even **1 epoch** made LR 0.2 trainable; 1 vs 5 epochs: no consistent difference; but **15 epochs was less robust** than 5 (4–5/5 seeds, 0.04–0.12 Å) while having the best unpruned RMSE (3.79 Å). **Partly supported** A plausible, untested reason: a longer warmup leaves fewer epochs at the large LR, and the large LR is what creates robustness.

E3: LR dose–response (all 7 warmup LRs, 5 seeds each). “Dose–response” is a term from medicine: if more dose gives steadily more effect, that is strong evidence of cause.

Warmup LR	0.005	0.01	0.02	0.03	0.05	0.1	0.2
Unpruned test RMSE	3.78	3.76	3.79	3.82	3.84	3.84	3.84
Δ at 70%	0.35	0.26	0.19	0.14	0.09	0.12	0.09
Δ at 90%	0.74	0.65	0.54	0.51	0.49	0.51	0.44

Robustness rises steadily with the LR (Spearman correlation between LR and Δ : -0.88 at 70%). Crucially, **LR 0.01 gave the best unpruned model but was far less robust than 0.2**, so robustness follows the LR, not the starting accuracy. This strengthens H3 beyond the two-point comparison, but because it was designed after seeing the results it counts as supporting evidence, not a formal test.

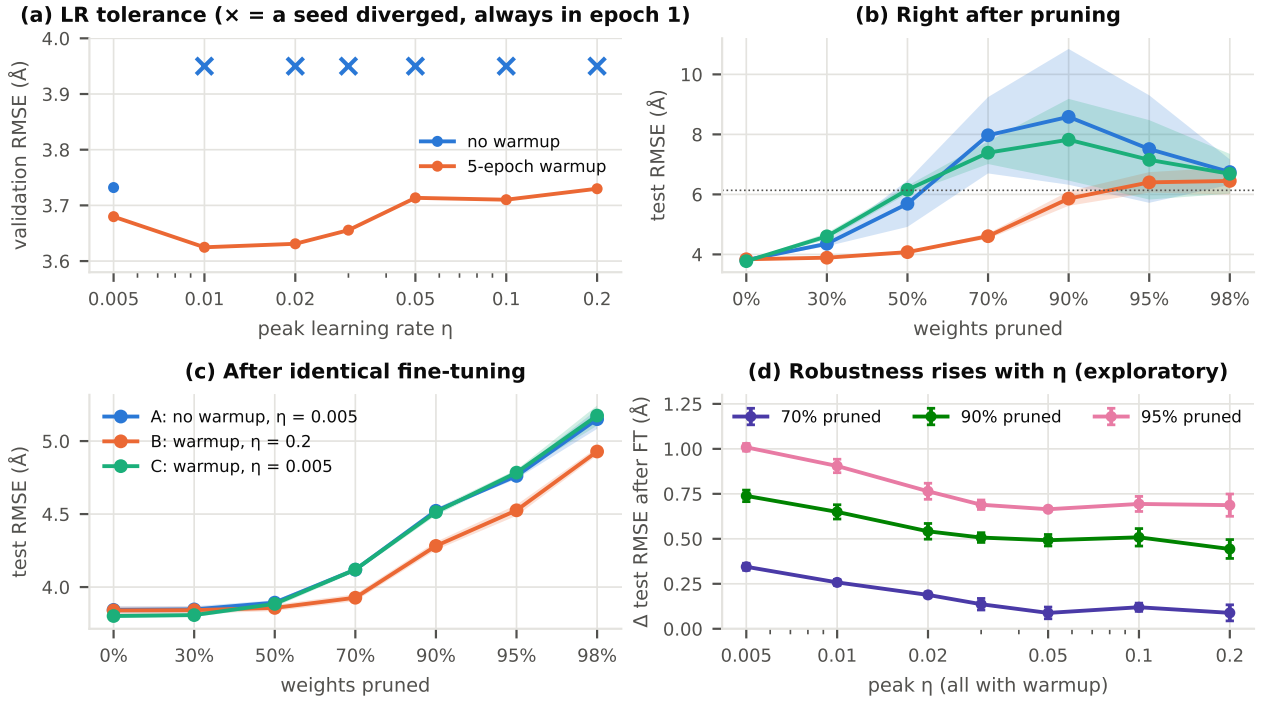


Figure 3: (a) LR tolerance: without warmup everything above 0.005 fails ($\times$). (b, c) Test RMSE vs sparsity for A, B, C before and after identical fine-tuning (the spec’s “accuracy vs sparsity” plot; lower is better; dotted = predicting the mean). (d) E3: Δ falls as the warmup LR rises.

5.10 Conclusion and limitations

Did we observe the effect? Yes. At matched unpruned accuracy, the warmup-trained network lost 0.22–0.27 Å less after pruning and fine-tuning (5/5 seeds), and far less before fine-tuning.

Do the results support the mechanism? Yes, on both links. Link 1: warmup raised the tolerable $\text{LR} \geq 20\times$ (H1). Link 2: the robustness comes from the larger LR, not from warmup itself (H3, with E3 showing a steady trend). E1 adds that the large LR also helps recovery. Warmup’s effect is therefore **indirect**.

Limitations: one tabular dataset and one MLP (the papers used CNNs on images); one-shot rather than iterative pruning; a single fit/validation split (seeds capture initialisation noise, not data-split noise); B’s LR sits at the top of the grid; large-LR training *without* warmup cannot be tested because it diverges; and the reason large-LR networks are robust is only explored post-hoc.

5.11 The spec’s other extensions (not done, but could be asked)

- **Timing** (prune during warmup vs well after it). *Expectation:* during warmup the network has barely trained, so weight magnitudes say little about importance; pruning well after warmup should be better. *Test:* prune at several epochs (e.g. 2, 5, 20, 100), same retraining budget, compare Δ .
- **Distillation.** A small “student” network is trained to copy a large “teacher’s” outputs. Question: does a warmup-trained teacher give a better student? *Test:* train teachers with and without warmup at matched accuracy, distil identical students, compare.
- **Neural architecture search (NAS).** Automatically searching over architectures (e.g. random search). Question: does the schedule used to *score* candidates during the search change which architecture wins, even if all are later retrained the same way? *Test:* run the same search twice, with and without warmup, compare the selected architectures after full retraining.

Cheat sheet: warmup and pruning

- **Chain:** warmup $\rightarrow$ larger trainable LR [3] $\rightarrow$ better pruning recovery [2]. Spec: test it.
- **Setup:** 4×256 ReLU, momentum 0.9, 100 epochs, cosine; warmup = 5 linear epochs. 5 seeds.
- **H1** supported: max stable LR 0.005 (no warmup) vs 0.1, trains at 0.2 ($\geq 20\times$). All failures in epoch 1.
- **Conditions** (rules fixed in advance): A no warmup 0.005; B warmup 0.2 (matched within $0.05 \text{ \AA}$); C warmup 0.005 (control).
- **Pruning:** one-shot global magnitude, biases kept, 0–98%. **Fine-tune:** identical, LR 0.001, 10 epochs, masks.
- Δ = RMSE after fine-tune – unpruned RMSE. Smaller = more robust.
- **H2** supported: B's Δ 0.22–0.27 $\text{\AA}$ smaller than A's at $\geq 70\%$, 5/5 seeds. Before FT at 70%: B 4.6 vs A 8.0 $\text{\AA}$.
- **H3** supported: $C \approx A$ (slightly worse); B beats C by 0.26–0.32 $\text{\AA}$. The LR is the cause.
- **Confounders ruled out:** dead units (B fewest), fine-tune LR (favours A/C), starting point (B slightly worse).
- **E1** LR rewinding: all recover better, A–B gap *widens* (contradicted). Large LR helps twice.
- **E2** warmup length: $1 \approx 5$ epochs; 15 less robust (partly supported).
- **E3** (exploratory): $\Delta_{70\%}$ 0.35 $\rightarrow$ 0.09 as LR 0.005 $\rightarrow$ 0.2; Spearman -0.88 .
- **Answer:** warmup improves pruning robustness *indirectly*, by unlocking a large LR.

5.12 Likely questions: warmup and pruning**Q34. What is learning-rate warmup and why does it help?**

Start with a very small LR and increase it to the target over the first steps or epochs. According to Kalra & Barkeshli, its main benefit is that the network tolerates a larger target LR: the small early steps move it from a sharp, unstable region into a flatter one before the large steps begin.

Q35. What is magnitude pruning? Global vs layer-wise? One-shot vs iterative?

Remove the weights with the smallest absolute values. Global ranks all weights together; layer-wise removes the same fraction from each layer. One-shot prunes once to the target sparsity; iterative prunes in small rounds with retraining in between. We used one-shot global pruning of the weight matrices.

Q36. Why must the unpruned networks have comparable accuracy?

So that any difference after pruning reflects robustness and not a better starting point. We required B's validation RMSE to be within $0.05 \text{ \AA}$ of A's, by a rule fixed in advance.

Q37. Why must the fine-tuning procedure be identical for both networks?

Otherwise a difference after fine-tuning could come from the fine-tuning recipe rather than from how the networks were trained. Same optimiser, LR, epochs and masks for all.

Q38. How did you separate the effect of warmup from the effect of a larger learning rate?

With the control C: warmup at A's small LR. C behaved like A, while B (warmup with a large LR) was much more robust, so the large LR is the cause and warmup is only the enabler.

Q39. Why did all no-warmup failures happen in the first epoch?

At initialisation the loss surface is sharp; full-size steps, amplified about $10\times$ by momentum, overshoot immediately and the loss explodes. Warmup's small first steps avoid exactly that moment.

Q40. Why do pruning masks need to be re-applied after every step?

A pruned weight still receives a gradient (and momentum), so one optimiser step would make it non-zero again. Multiplying by the mask keeps the network truly sparse.

Q41. Why measure the error both before and after fine-tuning?

Before shows the raw damage of removing weights; after shows how well the network recovers. The spec asks for the recovered accuracy; B was better on both.

Q42. Why use at least three seeds and report mean $\pm$ std?

One run can be lucky. Several seeds show how much results vary from initialisation and batch order, and paired comparisons per seed let us check that a difference is consistent and larger than the noise.

Q43. What is fine-tuning vs learning-rate rewinding, and what did you find?

Fine-tuning retraines the pruned network at a small fixed LR; LR rewinding retraines it with its original, large-LR schedule. Rewinding improved recovery for all networks and widened B's advantage, because B rewinds to LR 0.2.

Q44. What is the Lottery Ticket Hypothesis?

A large network contains a small subnetwork that, trained from the same initial weights, can match the full network's accuracy. It is found with iterative magnitude pruning; for deeper networks, warmup helped find such subnetworks.

Q45. How do you read an accuracy-vs-sparsity plot?

x-axis: fraction of weights removed; y-axis: error (for us, lower is better) or accuracy (higher is better). A robust network's curve stays flat for longer before it rises. Compare curves at matched starting points, look at the gap at high sparsity, and check the shaded spread across seeds.

Q46. Did your results support or contradict the proposed mechanism? What if they had contradicted it?

They supported it on both links (H1, H3, E3). If they had contradicted it, possible reasons would be: a small, over-parameterised MLP on tabular data is very different from the image CNNs in the papers; one-shot instead of iterative pruning; or the network being so redundant that pruning barely hurts any condition. An honest "contradicted" is a valid result.

Q47. What are the main threats to validity in your experiment?

Comparability of unpruned accuracy, dead units acting as free sparsity, the fine-tuning LR being a choice, one-shot pruning, a single fit/validation split, and only one dataset and architecture.

6 General research-method questions

These apply to any of the five datasets and are the most likely kind of test question.

Q48. What makes a good hypothesis?

It is stated *before* the experiment, makes a specific, measurable prediction (“at least one grid step higher”, “within 0.05 Å”), gives a reason based on theory, and says what result would contradict it.

Q49. What makes an experiment controlled and fair?

Change one variable at a time; keep everything else fixed; use the same data splits for every setting; repeat over several seeds; decide the measurements and decision rules in advance; and include a control when two factors are tangled together (our condition C).

Q50. Why should hypotheses and decision rules be written before running?

So the result cannot shape the prediction. Otherwise it is easy, even unintentionally, to choose thresholds or settings that make the outcome look good. We committed ours to git before running.

Q51. Your hypothesis was contradicted. Is the experiment a failure?

No. A contradicted hypothesis is a real finding if the experiment was fair. Explain *why* the reasoning was wrong: our “feature ceiling” assumption made several Part 1 hypotheses too pessimistic about capacity, and the architecture axis revealed it.

Q52. Why not tune hyperparameters on the test set?

The test set must be a fair, unseen final exam. Choosing settings by test score lets the test set influence the model, so its score becomes optimistic and no longer estimates real-world performance.

Q53. How do you decide whether a difference is meaningful?

Compare it with the noise (standard deviation across folds and seeds), and check that it points the same way in every paired seed. A gap smaller than the noise is not evidence.

Q54. What is the difference between correlation and causation in your study?

B being both large-LR and robust is a correlation. The control C (warmup without the large LR) and the dose-response (E3) were needed to argue that the LR *causes* the robustness.

Q55. How would you interpret a learning curve?

Training and validation loss per epoch. Both falling: still learning. Training falling, validation flat or rising: overfitting. Both high and flat: underfitting (too little capacity or too small an LR). Jagged: noisy updates, e.g. a constant LR with small batches. Explodes: LR too large.

Q56. Why is reproducibility important, and how did you achieve it?

Others (and you) must be able to repeat the result. Fixed seeds for splits, initialisation and batch order; results cached to CSV; all code in the notebook and `src/`; pinned package versions.

Q57. What would you do differently or next?

Group the split by protein if IDs were available; iterative pruning; k-fold instead of a single split in Part 2; test the mechanism behind large-LR robustness (flatness, weight concentration) directly; try the regularisation and initialisation axes.

Q58. How would your answers change for a classification dataset?

Stratify directly on the class labels; handle imbalance with class weights or resampling; evaluate with per-class precision, recall, F1 or AUC instead of plain accuracy; use cross-entropy loss with a sigmoid or softmax output.

7 Glossary

Term	Meaning
Å (ångström)	10^{-10} m. Unit of RMSD and of all our errors
Activation function	The non-linear function applied after each layer's weighted sum (sigmoid, tanh, ReLU)
Adam	Optimiser combining momentum with per-weight step scaling, plus bias correction
Backpropagation	Using the chain rule to compute every weight's gradient, from the loss backwards
Batch / mini-batch	The subset of rows (256) used for one gradient step
Bias (metric)	Mean of (prediction – truth) in a group; positive = guesses too high
Bias correction	Adam's fix for its averages starting at 0
Bimodal	A distribution with two peaks
Confounder	A hidden factor that could explain a result instead of the one you are testing
Control condition	A condition that isolates one factor (our C: warmup without the large LR)
Cosine decay	LR schedule following half a cosine wave from peak to 0
Cross-validation (k-fold)	Rotate which fold is scored; average the k scores
Data leakage	Information from evaluation data reaching the model during training or preprocessing
Dead ReLU	A unit that outputs 0 for every input and never recovers
Decoy	A candidate (predicted) 3D shape of a protein; one row of our data
Divergence	Training blowing up (loss becomes huge or NaN), usually from too large an LR
Dose–response	Effect growing steadily with the size of the cause; evidence of causation
Dropout	Randomly switching units off during training as regularisation
Early stopping	Stop when validation error stops improving; keep the best weights
Epoch	One pass over all training rows
Fine-tuning	Retraining a pruned network at a small fixed LR
Global pruning	Ranking all layers' weights together when pruning
He / Xavier init	Weight initialisations sized for ReLU / for sigmoid and tanh
IQR rule	Outlier if beyond $1.5 \times \text{IQR}$ outside the 25th–75th percentiles
Learning rate (LR, η)	Size of each optimiser step
Lottery Ticket Hypothesis	Large nets contain small subnetworks that train to full accuracy from the same initial weights
LR rewinding	Retraining a pruned network with its original LR schedule
Magnitude pruning	Removing the weights with the smallest $ w $
MAE / RMSE / R^2	Mean absolute error / root mean squared error / fraction of variance explained
Mask	0/1 grid marking which weights are pruned
Momentum	Running velocity of gradients; effective step $\approx \eta / (1 - \beta)$
Mutual information	Measures any (incl. non-linear) dependence between two variables
Overfitting	Fitting training data so closely that performance on unseen data gets worse
Paired comparison	Comparing two settings seed-by-seed on the same folds so shared noise cancels
Pre-registration	Writing hypotheses and rules down before running the experiment
Redundant feature	Informative, but its information is already in other features
Regression to the mean	Predictions pulled towards the average: low values over-, high values under-predicted
RMSD	The target: how far a decoy's atoms are from the true structure (Å)
RMSprop	Optimiser scaling each step by the recent gradient size (no bias correction in PyTorch)
Robustness (Δ)	Extra error after pruning and fine-tuning; smaller is more robust
Saturation	Sigmoid/tanh going flat for large inputs, so the gradient ≈ 0

Term	Meaning
Seed	Number fixing the randomness (initial weights, batch order)
Sparsity	Fraction of weights set to zero
Stratification	Making every split have the same target (or class) mix as the whole
Uninformative feature	No relationship with the target
Vanishing gradients	Gradients shrinking layer by layer going backwards, so early layers barely learn
Warmup	Ramping the LR up from near 0 to its peak at the start of training
Weight decay	Penalty on large weights (L2) as regularisation
Winsorise (clip)	Replace values beyond a limit by the limit; the row is kept
Z-score	$(x - \mu)/\sigma$: standardisation to mean 0, std 1
